

Entwicklerdokumentation

Inhaltsverzeichnis

1. Architecture Notebook	2
1.1. Zweck	2
1.2. Architekturziele	2
1.3. Annahmen und Abhängigkeiten	2
1.4. Architektur-relevante Anforderungen	3
1.5. Entscheidungen, Nebenbedingungen und Begründungen	3
1.6. Schichten oder Architektur-Framework	4
1.7. Architektursichten (Views)	4
2. Design	12
2.1. Zweck	12
2.2. Schnittstellen	12
2.3. Datenmodell	13
3. Vollständige Code-Übersicht	14
3.1. Überblick	14
3.2. Projektstruktur	14
3.3. Backend (Server)	15
3.4. Frontend (Browser-App)	17
3.5. Docker/Dev-Skripte (kurz)	20
4. Setup und Guidelines	22
4.1. Docker Guide	22
4.2. Prisma ORM Guide	25
4.3. Supabase Guide	26
4.4. ESLint Leitfaden (Frontend & Backend)	28
4.5. Styleguide	30
5. Integration und Deployment	37
5.1. Integration	37
5.2. Deployment	39

Link zum Repository: <https://github.com/1tristan0/SE-HabitTracker-3.B>

1. Architecture Notebook

1.1. Zweck

Dieses Dokument beschreibt die Philosophie, Entscheidungen, Nebenbedingungen, Begründungen, wesentliche Elemente und andere übergreifende Aspekte des Systems, die Einfluss auf Entwurf und Implementierung haben.

1.2. Architekturziele

Die Applikation basiert auf einer klassischen Dreischicht-Architektur (Client, Logik, Datenspeicherung) und soll auf verschiedensten Endgeräten (Desktop und Mobil) über den Browser bedienbar sein. Dafür wird ein leichtgewichtiges und responsives Frontend mit React entwickelt, das mit Tailwind CSS und Bootstrap gestaltet wird.

Das Backend wird mit Node.js und Express.js umgesetzt. Die Kommunikation zwischen Frontend und Backend erfolgt über eine RESTful API. Als Datenbank nutzen wir Supabase (PostgreSQL) mit eingebautem Authentifizierungsdienst.

- **Plattformunabhängigkeit:** Die Anwendung muss sowohl auf mobilen Geräten als auch auf Desktop-Browsern voll funktionsfähig sein.
- **Erweiterbarkeit:** Die Struktur soll zukünftige Erweiterungen (z.B. Gruppenfunktionen oder Gamification) einfach ermöglichen.
- **Schnelle Entwicklungszyklen:** Durch Nutzung moderner Web-Technologien sollen Entwicklungs- und Deployment-Prozesse effizient gehalten werden.
- **Datensicherheit und Benutzerverwaltung:** Supabase übernimmt die Authentifizierung sowie Speicherung personenbezogener Daten.
- **Geringe Einstiegshürde für Entwickler:** Vertraute Frameworks wie React und Express.js ermöglichen eine einfache Einarbeitung.

1.3. Annahmen und Abhängigkeiten

- Das Projekt wird von einem studentischen Team mit begrenzter Erfahrung im Bereich Webentwicklung umgesetzt.
- Supabase wird als externer Dienst verwendet, inklusive Authentifizierungs- und Datenbankfunktionalität.
- Die Nutzer benötigen ein internetfähiges Gerät mit aktuellem Browser.
- Die Deployment-Umgebung ist ein von der HTW Dresden bereitgestellter Server (z.B. iseproject02.informatik.htw-dresden.de).
- Es gibt keine Abhängigkeiten zu Legacy-Systemen.
- Zeitliche Ressourcen sind begrenzt.
- Das Budget ist auf Open-Source-Technologien und kostenlose Dienste beschränkt.

1.4. Architektur-relevante Anforderungen

- Nutzerregistrierung und -anmeldung (Supabase Auth)
- Erstellen, Bearbeiten und Löschen von Habits
- Anzeige von Habit-Statistiken
- Datenpersistenz (Supabase PostgreSQL)
- Responsives Design

1.5. Entscheidungen, Nebenbedingungen und Begründungen

1. Verwendung von React im Frontend

- Ermöglicht komponentenbasiertes UI-Design und hohe Wiederverwendbarkeit. 2 Teammitglieder haben bereits erste Vorerfahrungen mit der Verwendung von React in der Webentwicklung
- Gängige Wahl für moderne Webanwendungen, große Community und viele Ressourcen verfügbar.

2. Einsatz von Tailwind CSS und Bootstrap

- Ermöglicht schnelles Prototyping und konsistente Gestaltung mit geringem Designaufwand.
- Ermöglicht responsives Design und einfache Anpassung an verschiedene Bildschirmgrößen. (App soll auf Desktop und Mobil funktionieren)

3. Backend mit Node.js und Express.js

- Einfache Integration mit JavaScript-basiertem Frontend, große Entwickler-Community.
- Hohe Performance und Skalierbarkeit.
- Vorerfahrung der Team-Mitglieder mit Node.JS und Express.js

4. Nutzung von Supabase (PostgreSQL) und Prisma ORM

- Spart Zeit durch fertige Authentifizierungslösungen und Datenbankverwaltung in der Cloud.
- Team muss keine eigene Lösung zum Hosting einer Datenbank finden.
- Verwendung eines ORM ermöglicht komplexere Features mit geringerem Aufwand umzusetzen sowie kein reines SQL im Backend verwalten zu müssen
- Prisma Schema verringert den Aufwand, Anpassungen des Domänenmodells auf DB-Schema zu übertragen (Objektorientiert)

5. Dreischicht-Modell

- Getrennte Verantwortlichkeiten, einfache Skalierung, klare Trennung zwischen UI, Logik und Datenspeicherung.

1.6. Schichten oder Architektur-Framework

ENTWURF DER ARCHITEKTUR: Die Anwendung nutzt ein klassisches Dreischichten-Framework (Three-Tier-Architektur): Presentation, Logic und Data.

Schicht	Beschreibung
Präsentationsschicht (Client)	• Leichtgewichtiges, responsives Frontend in React • Styling mit Tailwind CSS & Bootstrap • Kommuniziert per HTTP/REST mit der Logikschicht
Logikschicht (Server)	• Node.js + Express.js als API-Backend • Implementierung aller Geschäftsprozesse (Use Cases) • Authentifizierung via Supabase Auth über Middleware • Mapping von HTTP-Requests auf Domänen-Services • DB-Abfragen/Operationen werden durch den Prisma Client ausgeführt
Datenhaltungsschicht (Database)	• Supabase (PostgreSQL) für persistente Datenspeicherung • PostgreSQL für CRUD-Operationen • Supabase Auth und Policies zur Zugriffskontrolle

1. **Kommunikation:** – RESTful API zwischen Präsentations- und Logikschicht – Prisma Client verknüpft Logik- und Datenhaltungsschicht
2. **Prinzipien:** – Jede Schicht nur über wohldefinierte Schnittstellen (Top-Down). – Domänenschicht (innerhalb der Logikschicht) bleibt Framework-unabhängig. – Querschnittsfunktionen (Logging, Fehlerbehandlung, Konfiguration) werden zentral in der Logikschicht bereitgestellt.

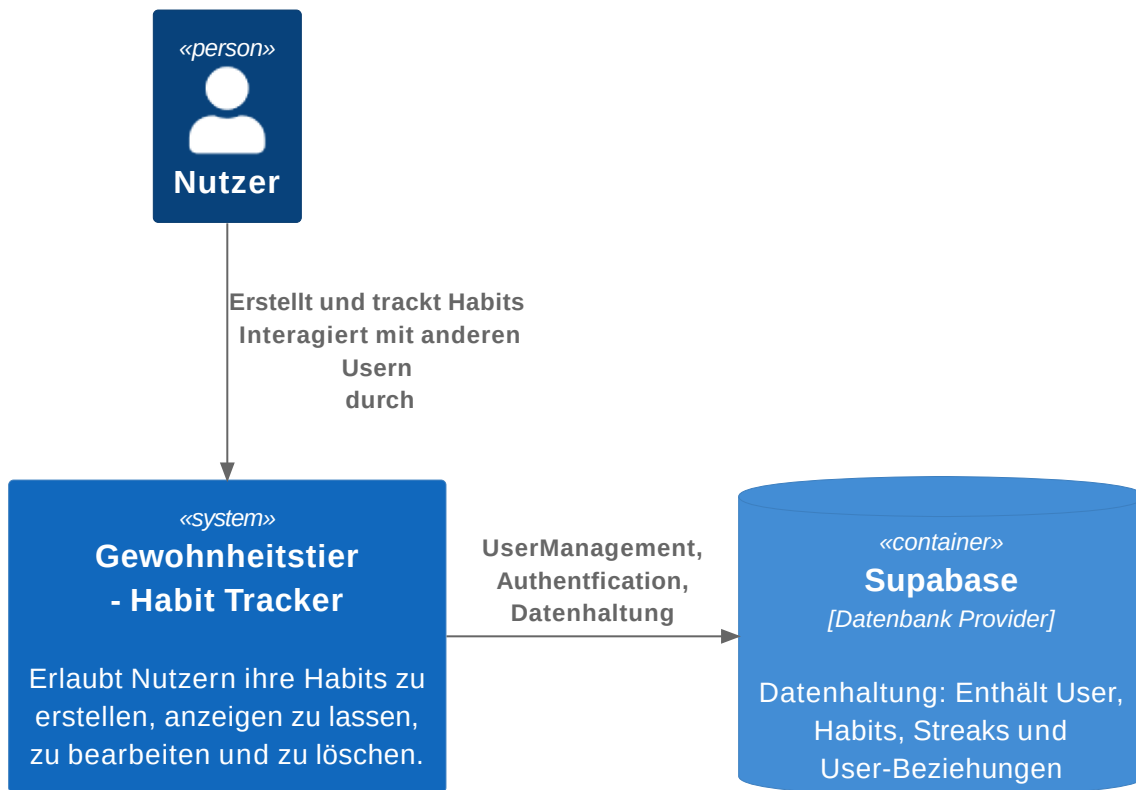
1.7. Architektursichten (Views)

Die Architektur folgt einem **dreischichtigen Aufbau**:

- **Präsentationsschicht (Frontend):** React, Tailwind CSS, Bootstrap – Darstellung der Benutzeroberfläche und UX.
- **Logikschicht (Backend):** Node.js mit Express.js – Verarbeitung von Geschäftslogik, Validierung, API-Endpunkte.
- **Datenhaltungsschicht:** Supabase/PostgreSQL – Speicherung und Abruf aller Nutzerdaten, Habits und Metadaten.

Der Austausch zwischen den Schichten erfolgt über eine klar definierte REST-API. Die Architektur ist bewusst modular aufgebaut, um spätere Komponenten wie Push-Benachrichtigungen, Kalenderintegration o.ä. ergänzen zu können.

1.7.1. Kontextabgrenzung

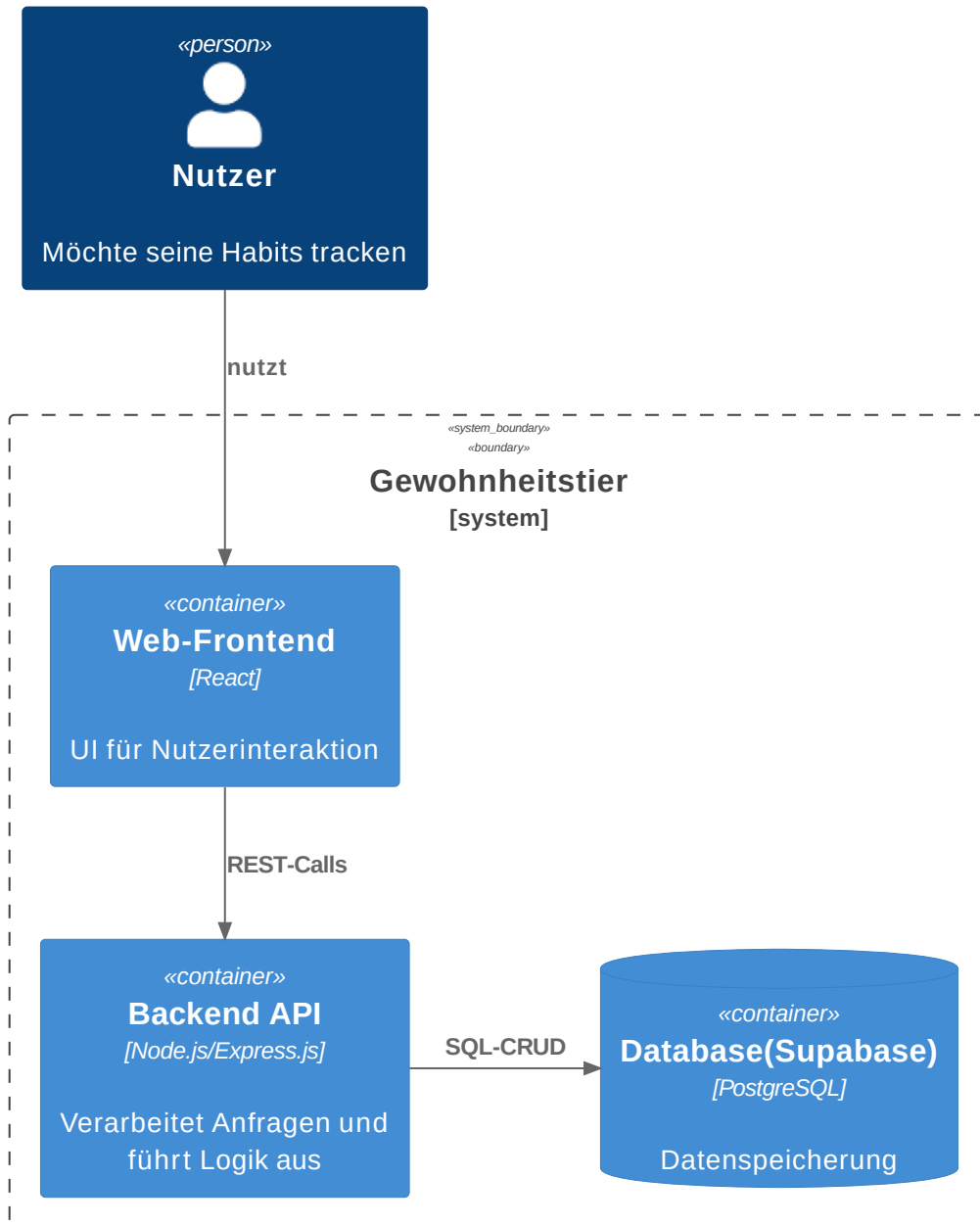


Erklärung:

Unsere App "Gewohnheitstier" ist ein einfacher Habit-Tracker:

- **Nutzer:** Legt in der App neue Gewohnheiten (Habits) an und trackt deren Fortschritt.
- **Gewohnheitstier (System):** Bietet Funktionen zum Erstellen, Anzeigen, Bearbeiten und Löschen von Habits.
- **Supabase (externer Container):** Übernimmt User-Management, Authentifizierung und persistente Datenspeicherung (Nutzer, Habits, Streaks, Nutzer-Beziehungen).

1.7.2. Container

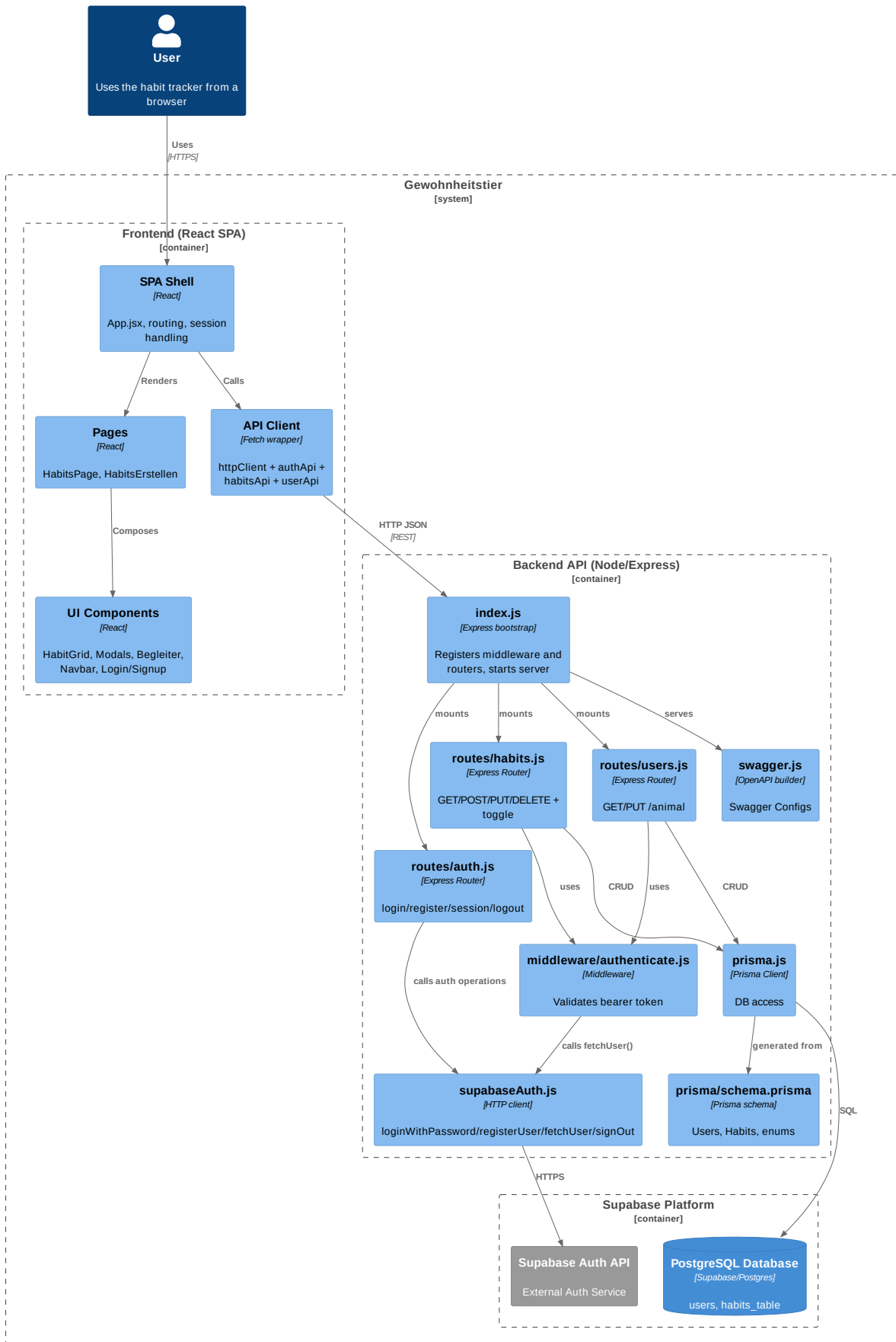


Erklärung:

- **Nutzer:** Möchte seine Habits über Browser oder Mobile App verwalten.
- **Web-Frontend** (React): Browser-UI für Nutzerinteraktion; kommuniziert per REST mit der Backend API.
- **Backend API** (Node.js/Express): Verarbeitet HTTP/HTTPS-Anfragen, enthält Geschäftslogik und führt SQL-CRUD-Operationen auf die Datenbank aus.
- **Database** (Supabase/PostgreSQL) : Übernimmt Authentifizierung, User-Management und persistente Datenspeicherung von Nutzern, Habits, Streaks und Beziehungen.

1.7.3. Components

Level 3 im C4-Modell folgt in späteren Iterationen

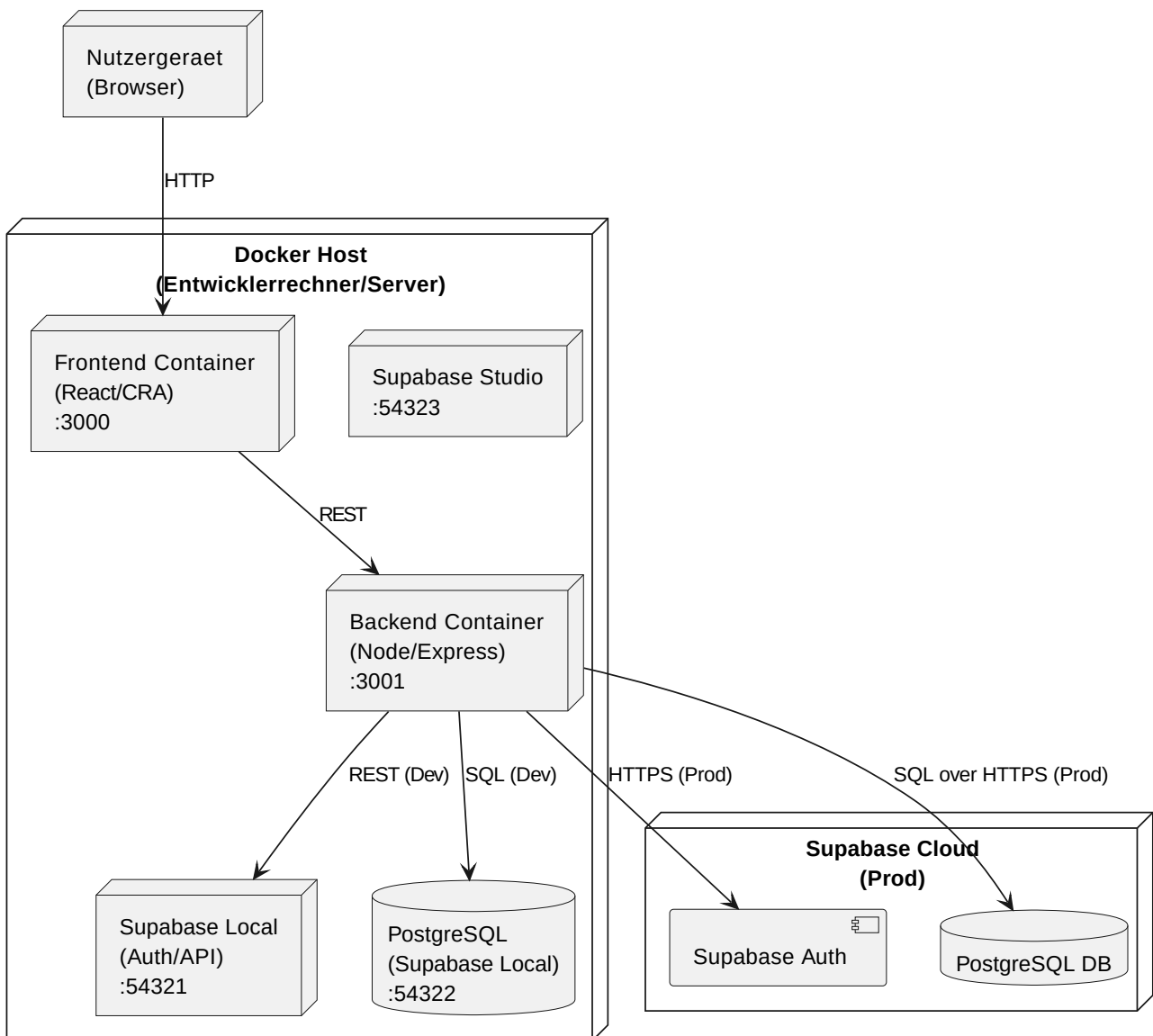


Erklärung:

- **C3-Diagramm (Komponentensicht):** Zeigt die wichtigsten Komponenten innerhalb des **Backend API-Containers** und deren Zusammenspiel.
- **Controller/Routen:** Endpunkte der REST-API, die HTTP-Requests entgegennehmen und an die Geschäftslogik weiterleiten.
- **Services:** Kapseln die Geschäftslogik (Use Cases) und koordinieren die Verarbeitung.
- **Repositorys/Adapter (Prisma Client):** Verantwortlich für den Datenzugriff auf die Supabase/PostgreSQL-Datenbank.
- **Querschnittsfunktionen:** Authentifizierung (Supabase Auth), Validierung und Fehlerbehandlung sind klar sichtbar.
- **Abhängigkeitsrichtung:** UI → API → Service → Datenzugriff.

1.7.4. Verteilungsdiagramm (Deployment Diagramm)

Deployment Diagramm - Gewohnheitstier

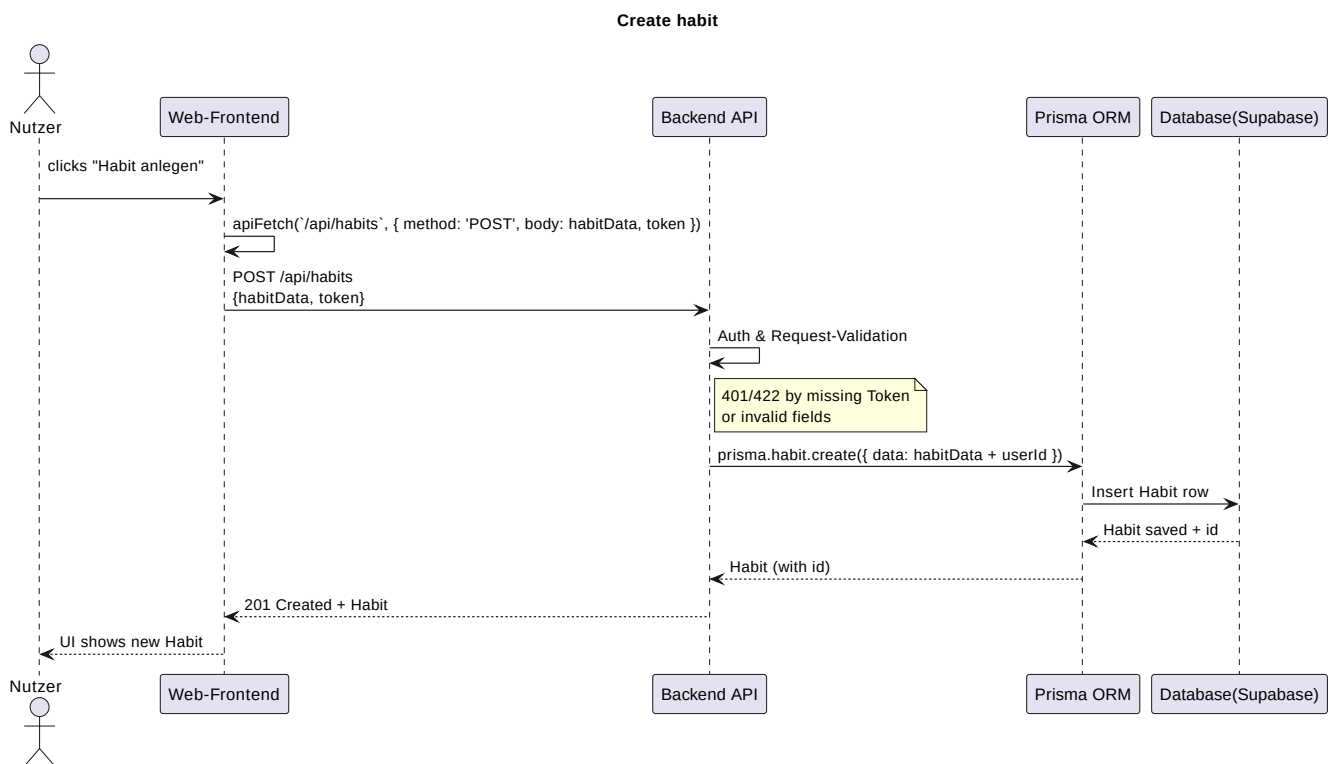


Erklärung:

- **Nutzergerät (Browser):** Öffnet das Frontend im Browser und kommuniziert über HTTP/REST.
- **Docker Host:** Führt die Container für Frontend (React/CRA), Backend (Node/Express) und die lokale Supabase-Instanz aus.
- **Supabase Local (Dev):** Stellt Auth/API, PostgreSQL und Studio lokal bereit (Ports 54321-54323).
- **Supabase Cloud (Prod):** Externer Dienst für Authentifizierung und PostgreSQL-Datenhaltung in der Produktionsumgebung.
- **Kommunikation:** Browser → Frontend → Backend per HTTP/REST; Backend → Supabase lokal (Dev) bzw. Supabase Cloud (Prod).

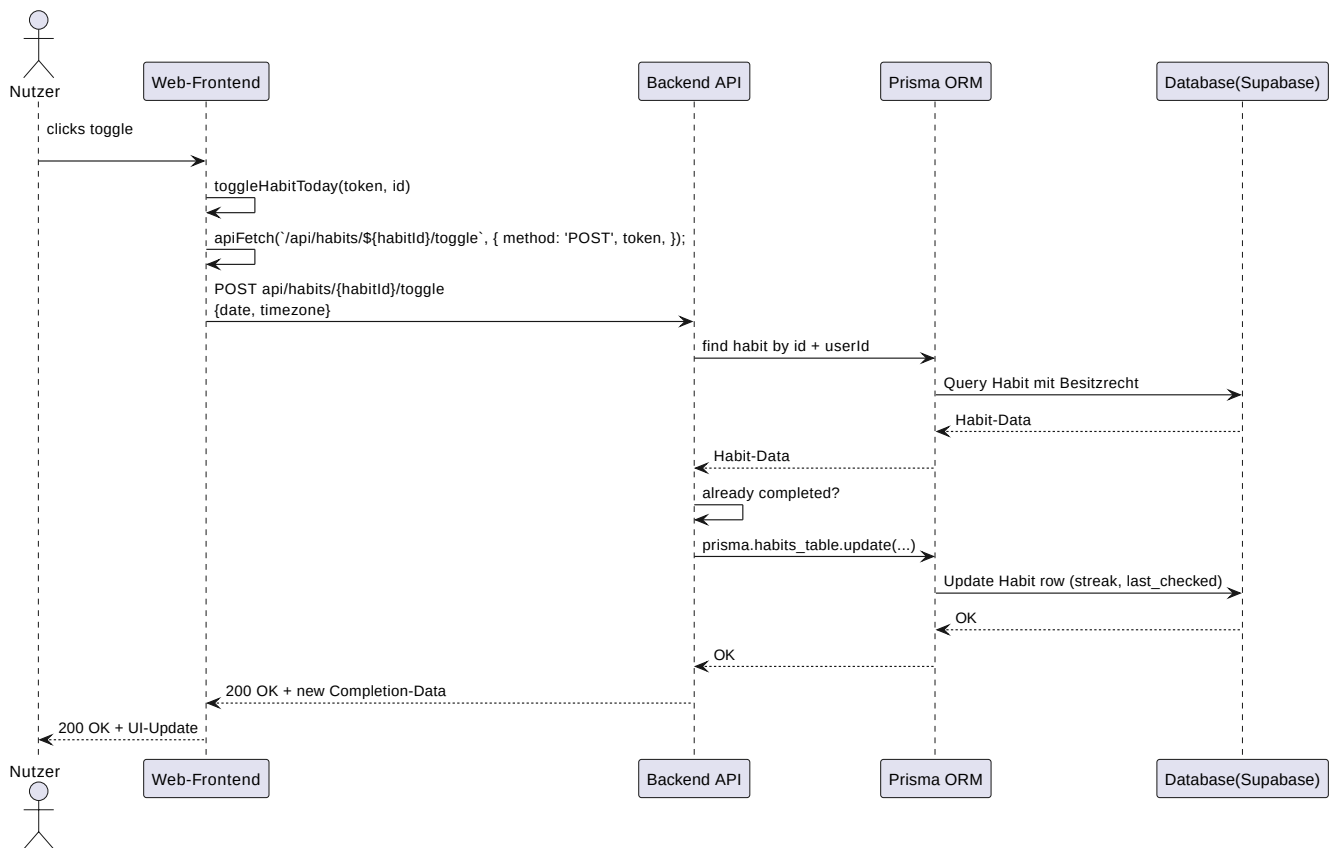
1.7.5. Sequenzdiagramme

Sequenzdiagramm Habit anlegen

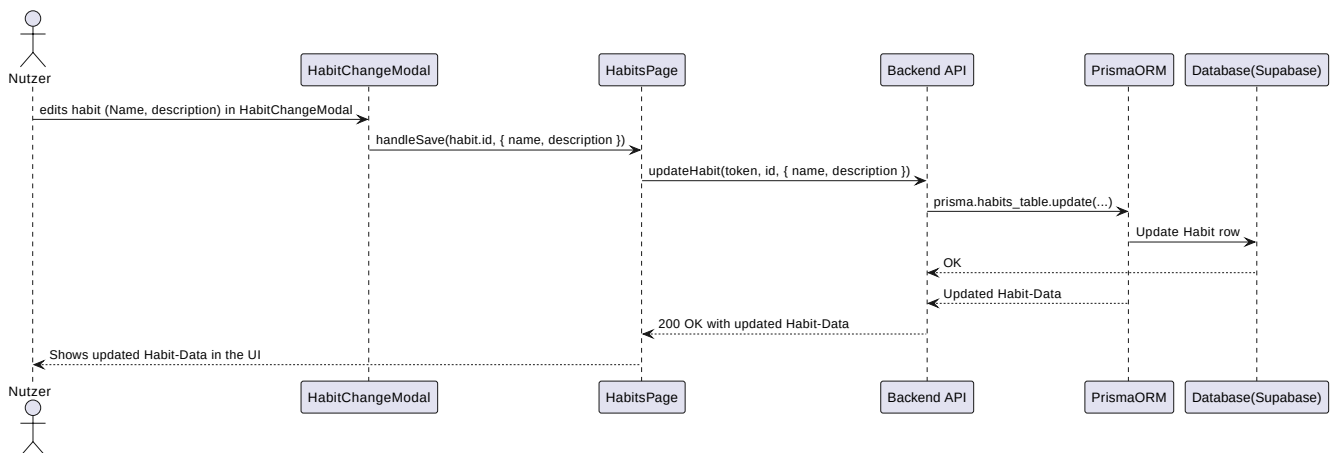


Sequenzdiagramm Habit abhaken

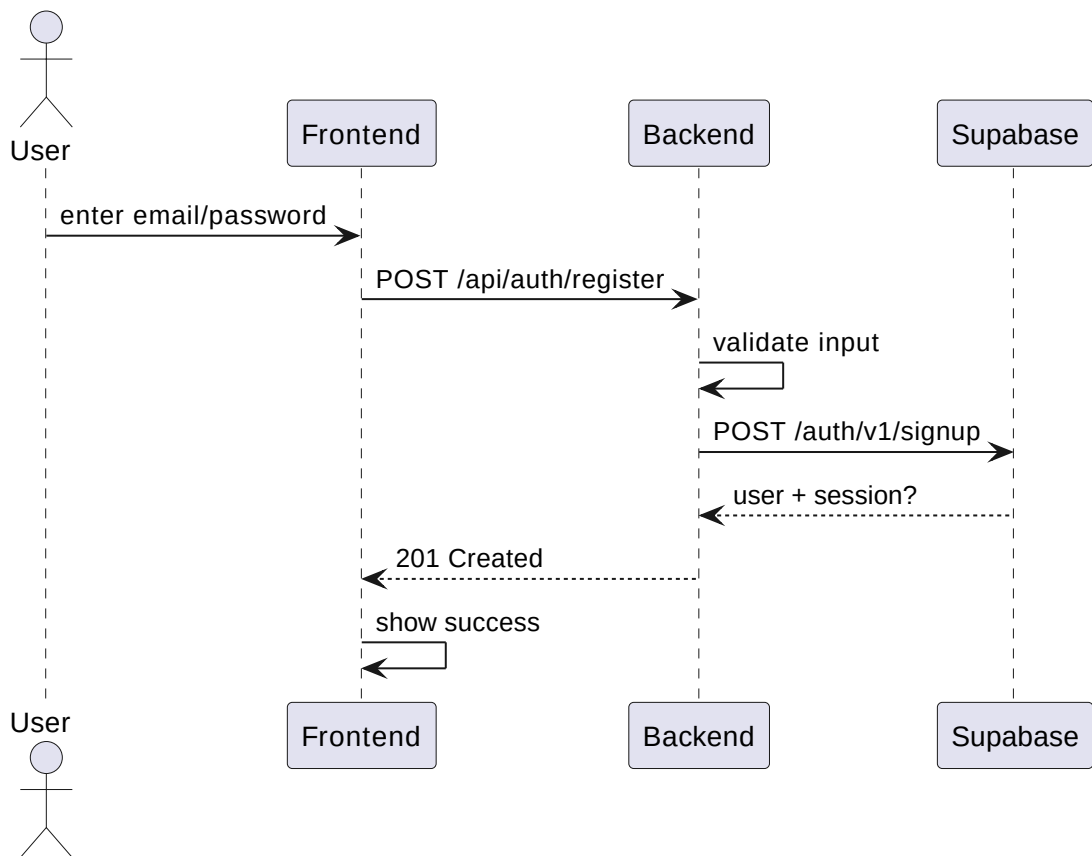
Habit abhaken (als erledigt markieren)



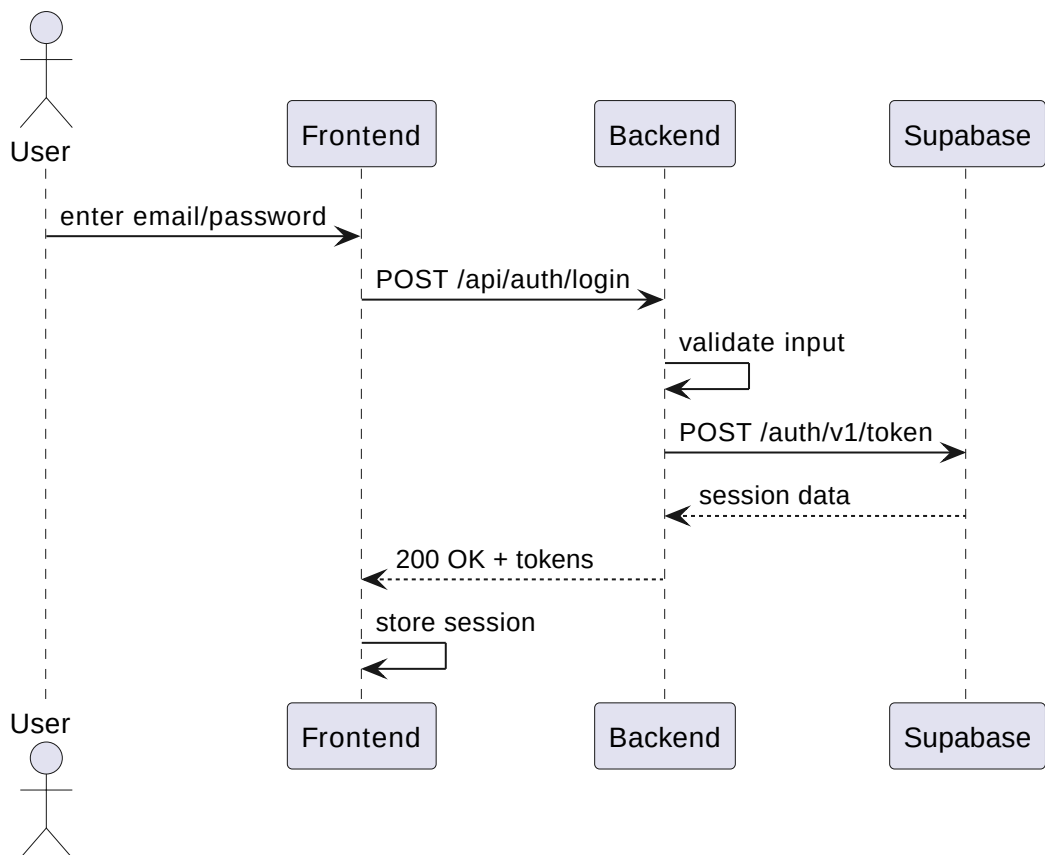
Sequenzdiagramm Habit bearbeiten



Sequenzdiagramm User registrieren



Sequenzdiagramm user log in



2. Design

2.1. Zweck

Dieses Dokument beschreibt wesentliche Entwurfsentscheidungen auf Ebene der Komponenten und Module. Diese dienen als Vorgabe für die Implementierung und erleichtern das Systemverständnis.

2.2. Schnittstellen

Frontend-Backend: Das Frontend kommuniziert mit dem Backend über eine Rest-API Schnittstelle: Folgende Endpunkte sind definiert:

2.2.1. Arbeit mit Gewohnheiten:

- PUT `/api/habits/${habitId}/check` - PUT-Anfrage an die API senden, um das Feld `last_checked` auf das aktuelle Datum zu setzen
- GET `/api/habits` - GET-Anfrage an die API senden, um alle Gewohnheiten abzurufen
- POST `/api/habits` - POST-Anfrage an die API senden, um eine neue Gewohnheit zu erstellen
- DELETE `/api/habits/${habitId}` - DELETE-Anfrage an die API senden, um eine Gewohnheit zu löschen
- POST `/api/habits/${habitId}/toggle` - POST-Anfrage an die API senden, um die Streak zu aktualisieren

2.2.2. Arbeit mit Gewohnheitstieren:

- GET `/api/users/animal` - GET-Anfrage an die API senden, um das Tier des Benutzers abzurufen
- PUT `/api/users/animal` - PUT-Anfrage an die API senden, um das Tier des Benutzers zu aktualisieren

2.2.3. Login

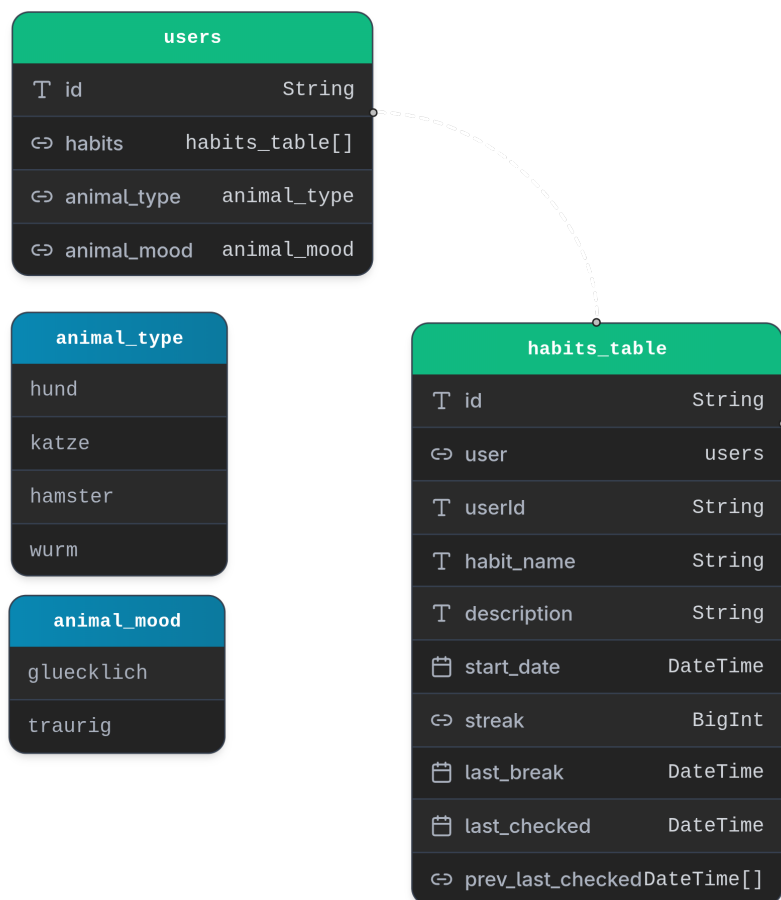
- POST `/api/auth/login` - POST-Anfrage an die API senden, um einen Benutzer zu authentifizieren und ein JWT-Token zu erhalten
- POST `/api/auth/register` - POST-Anfrage an die API senden, um einen neuen Benutzer zu registrieren
- POST `/api/auth/session` - POST-Anfrage an die API senden, um die aktuelle Sitzung zu überprüfen
- POST `/api/auth/logout` - POST-Anfrage an die API senden, um den Benutzer abzumelden
- Backend-Datenbank: Das Backend greift per Prisma ORM auf die Datenbank zu

Bei laufendem Docker-Container können die Schnittstellen unter <http://localhost:3001/api/docs/> nachvollzogen werden.

2.3. Datenmodell

Das Datenmodell wurde nach Vorgaben des Domänenmodell abgeleitet und entworfen. Die Definition der Datentypen, -namen sowie -relationen erfolgt innerhalb der Logikschicht unter Nutzung des Prisma Schemata (siehe [src/backend/prisma/schema.prisma](#)). Prisma erzeugt aus diesem Schema den ORM-Client für CRUD-Operationen. Außerdem generiert Prisma unter Beachtung des Schemata Migrationen und wendet diese auf die PostgreSQL Instanz von Supabase an, lokal sowohl als auch in der Cloud.

Das folgende UML-Diagramm stellt das interne Datenmodell dar:



3. Vollständige Code-Übersicht

3.1. Überblick

Kerngedanke: Der Browser spricht nur mit unserem eigenen Server. Der Server regelt Anmeldung und Datenbankzugriffe; die Datenbank liegt bei Supabase (Postgres). Supabase-Auth wird nur serverseitig genutzt, nie direkt im Browser.

3.2. Projektstruktur

```
src/
├── docker-compose.yml
├── docker-compose.dev.yml
├── docker-compose.prod.yml
├── dev-start.sh / dev-start.ps1
├── dev-db-setup.sh / dev-db-setup.ps1
├── backend/
│   ├── .env / .env.local
│   ├── Dockerfile
│   ├── index.js
│   ├── package.json / package-lock.json
│   ├── eslint.config.mjs
│   ├── __tests__/
│   │   ├── prismaHabits.unit.test.js
│   │   ├── prismaUsers.unit.test.js
│   │   ├── routesAuth.test.js
│   │   ├── routesHabits.test.js
│   │   ├── routesUsers.test.js
│   │   ├── smoke.test.js
│   │   └── supabaseAuth.test.js
│   ├── prisma/
│   │   ├── schema.prisma
│   │   └── migrations/
│   ├── prisma.config.js
│   └── src/
│       ├── prisma.js
│       ├── supabaseAuth.js
│       ├── swagger.js
│       ├── middleware/authenticate.js
│       └── routes/{auth.js, habits.js, users.js}
└── frontend/
    ├── .env
    ├── Dockerfile
    ├── package.json / package-lock.json
    └── src/
        ├── App.jsx / App.css
        ├── index.js / index.css
        └── setupTests.js / reportWebVitals.js / logo.svg
```



```

|—— api/{httpClient.js, authApi.js, habitsApi.js, userApi.js}
|—— hooks/useHabits.js
|—— pages/{HabitsPage.jsx, CreateHabitPage.jsx}
|—— components/
|   |—— auth/{Login.jsx, Signup.jsx}
|   |—— layout/Navbar.jsx
|   |—— habits/{HabitGrid.jsx, HabitCard.jsx, HabitCreateContent.jsx,
HabitForm.jsx, HabitList.jsx}
|       |—— habits/modals/{HabitChangeModal.jsx, HabitInfoModal.jsx}
|       |—— habits/calendar/Calender.jsx
|       |—— habits/calendar/modals/CalendarModal.jsx
|       |—— animal/Animal.jsx
|       |—— animal/modals/AnimalModal.jsx
|       |—— icons/{DeleteIcon.jsx, EditIcon.jsx}
|       |—— ui/
|           |—— Buttons/{CalendarDayButton.jsx, CloseButton.jsx,
SaveButton.jsx}
|               |—— InputField.jsx / Modal.jsx / ProgressCircle.jsx /
StreakBadge.jsx / Titel.jsx / ToggleSwitch.jsx
|               |—— lib/{convert.js, calendar.js, habit.js}
|               |—— __tests__/
|                   |—— Animal.test.js
|                   |—— AnimalModal.test.js
|                   |—— CalendarModal.test.js
|                   |—— Calender.test.js
|                   |—— HabitCard.test.js
|                   |—— HabitChangeModal.test.js
|                   |—— HabitCreateContent.test.js
|                   |—— HabitGrid.test.js
|                   |—— HabitInfoModal.test.js
|                   |—— Login.test.js
|                   |—— Navbar.test.js
|                   |—— Signup.test.js
|                   |—— calendar.test.js

```

3.3. Backend (Server)

Der Server läuft mit Node.js/Express. Er hat drei Aufgaben: (1) Nutzer registrieren/anmelden/abmelden, (2) Gewohnheiten verwalten, (3) User-Status (Begleiter) laden/setzen. Die Datenbank liegt bei Supabase (Postgres). Supabase-Auth wird nur vom Server angesprochen.

3.3.1. index.js

- Startet den Server, erlaubt Browser-Aufrufe (CORS) und verarbeitet JSON-Anfragen.
- **GET /** → kurzer "API läuft"-Text.
- Leitet Anfragen an:
 - **/api/auth** (Anmeldung/Registrierung/Session/Logout)

- `/api/habits` (Gewohnheiten)
- `/api/users` (Tier-Typ/Stimmung)
- `/api/docs` (Swagger UI)
- Port: `PORT` aus der Umgebung (in `backend/.env` auf 3001 gesetzt), Fallback 5000.

3.3.2. prisma.config.js

- Verweist auf `prisma/schema.prisma` und `prisma/migrations`.
- Nutzt `DATABASE_URL` aus der Umgebung.
- Prisma-Client geht auf den Standard-Output von `@prisma/client` (kein eigenes `output`).

3.3.3. prisma/schema.prisma

- Enums: `animal_type` (hund/katze/hamster/wurm), `animal_mood` (gluecklich/traurig).
- `habits_table`:
 - `id` (String mit `uuid()` Default)
 - `userId` (String, FK auf `users.id`)
 - `habit_name`, `description`, `start_date`, `streak` (BigInt), `last_break`, `last_checked`, `prev_last_checked` (Array)
- `users`:
 - `id` (String mit `uuid()` Default, über Trigger aus Auth-Tabelle übernommen)
 - `animal_type`, `animal_mood`
 - 1-n Relation zu `habits_table`

3.3.4. src/prisma.js

- Erstellt und exportiert `PrismaClient` mit Logging.

3.3.5. src/supabaseAuth.js

- Spricht die Supabase-Auth-REST-API an.
- Braucht `SUPABASE_URL` und `SUPABASE_ANON_KEY`.
- Kernfunktionen: `loginWithPassword`, `registerUser`, `fetchUser`, `signOut`.

3.3.6. src/middleware/authenticate.js

- Middleware für geschützte Routen.
- Erwartet Header `Authorization: Bearer <token>`.
- Prüft Token per Supabase und hängt `req.auth.user` an (sonst 401/403).

3.3.7. src/routes/auth.js

- JSON-Schnittstellen für Auth:
 - `POST /api/auth/login`
 - `POST /api/auth/register`
 - `POST /api/auth/session`
 - `POST /api/auth/logout`

3.3.8. src/routes/habits.js

- Geschützt durch `authenticate`.
- Helfer:
 - `serializeHabit` (Dates + BigInt sauber nach JSON)
 - `normalizeDateArray` (stellt Date-Arrays sicher)
- Endpunkte:
 - `GET /api/habits`: lädt alle Habits des Users; setzt `streak` auf 0, wenn `last_checked` > 48h her ist (und schreibt `last_break`).
 - `POST /api/habits`: legt Habit an (Startdatum heute, Streak 0, `prev_last_checked` leer).
 - `PUT /api/habits/:id`: aktualisiert Name/Beschreibung/Startdatum.
 - `DELETE /api/habits/:id`: löscht Habit.
 - `POST /api/habits/:id/toggle`: schaltet "heute erledigt"; aktualisiert `streak`, `last_checked`, `prev_last_checked`.

3.3.9. src/routes/users.js

- Geschützt durch `authenticate`.
- `GET /api/users/animal`: liefert `animal_type` und `animal_mood` des Nutzers.
- `PUT /api/users/animal`: setzt `animal_type` und/oder `animal_mood` (validiert gegen Enums).

3.3.10. src/swagger.js

- Konfiguriert Swagger/OpenAPI (Schemas für Auth, Habit, UserAnimal).
- UI ist unter `/api/docs` verfügbar.

3.3.11. Tests (`_tests_`)

- Unit- und Routen-Tests für Auth, Habits, Users, Prisma-Zugriffe und Supabase-Auth.

3.4. Frontend (Browser-App)

Die React-Anwendung läuft im Browser. Sie zeigt Login/Registrierung und danach die Gewohnheiten. Sie spricht ausschließlich mit dem eigenen Backend; Supabase-Schlüssel bleiben verborgen.

3.4.1. App.jsx

- Oberste React-Komponente mit React Router.
- Lädt gespeicherte Session und prüft Token per `/api/auth/session`.
- Routen:
 - `/` → `HabitsPage.jsx`
 - `/add` → `CreateHabitPage.jsx`

3.4.2. api/httpClient.js

- Hilfsfunktion für HTTP-Requests.
- Basis-URL: `REACT_APP_API_BASE_URL` (Repo-`.env` ist `http://localhost:3001`), Fallback `http://localhost:5000`.
- Setzt JSON-Header und optional `Authorization: Bearer <token>`.

3.4.3. api/authApi.js

- Backend-Calls: `login`, `register`, `fetchSession`, `logout`.
- Lokale Session: `persistSession`, `loadSession`, `clearSession`.

3.4.4. api/habitsApi.js

- Calls für Gewohnheiten: `fetchHabits`, `addHabit`, `updateHabit`, `deleteHabit`, `toggleHabitToday`.

3.4.5. api/userApi.js

- Calls für User-Begleiter: `fetchAnimal`, `setAnimal`.

3.4.6. pages/HabitsPage.jsx

- Seite nach erfolgreichem Login.
- Lädt Habits und Begleiterdaten; aktualisiert Habits bei Toggle/Edit/Delete.
- Rendert Habit-Grid, Info-/Change-Modal, Kalender sowie Begleiter-UI.

3.4.7. pages/CreateHabitPage.jsx

- Seite zum Anlegen neuer Gewohnheiten.
- Nutzt `HabitCreateContent` und ruft `addHabit` über das Backend auf, wenn Formular abgesendet wird.

3.4.8. hooks/useHabits.js

- Wiederverwendbarer Hook für Habit-Listen (load/add/remove/update).
- Existiert und ist funktionsfähig, wird derzeit aber in den Seiten nicht genutzt.

3.4.9. components

- Layout:
 - [Navbar.jsx](#)
 - zeigt Logo, Titel, Links zu HabitsPage und CreateHabitPage, sowie Logout-Button.
- Auth:
 - [Login.jsx](#)
 - zeigt Login-Formular.
 - [Signup.jsx](#)
 - zeigt Registrierungs-Formular.
- Habits:
 - [HabitGrid.jsx](#)
 - zeigt Raster aller Habits mit Toggle-Buttons.
 - [HabitCard.jsx](#)
 - zeigt einzelne Habit-Karte mit Name, Streak, Edit/Delete-Icons.
 - [HabitCreateContent.jsx](#)
 - zeigt Formular zum Anlegen neuer Gewohnheiten + Tipps.
 - [HabitForm.jsx](#) (aktuell nicht verknüpft)
 - [HabitList.jsx](#) (aktuell nicht verknüpft)
 - [HabitChangeModal.jsx](#)
 - zeigt Modal zum Ändern von Habit-Details ⇒ Name und Beschreibung können geändert werden.
 - [HabitInfoModal.jsx](#)
 - zeigt Modal mit Habit-Statistiken (Streak, Startdatum,...).
- Kalender:
 - [Calender.jsx](#)
 - zeigt Monatsübersicht mit Habit-Erledigungen(als Fortschrittskreis).
 - [CalendarModal.jsx](#)
 - zeigt Modal mit Details zu einem Kalendertag (erledigte/nicht erledigte Habits).
- Begleiter:
 - [Animal.jsx](#)
 - zeigt das Begleiter-Tier mit Stimmung und öffnet AnimalModal beim Klick zum Ändern des Tiers.
 - [AnimalModal.jsx](#)
 - zeigt Modal zum Auswählen des Tiers.
- UI:

- [ToggleSwitch.jsx](#)
 - Umschalt-Schalter-Komponente (für Habit "heute erledigt").
- [Modal.jsx](#)
 - generische Modal-Komponente, welche in allen Modals wiederverwendet wird
- [ProgressCircle.jsx](#)
 - zeigt Fortschrittskreis (für Kalender-Übersicht).
- [StreakBadge.jsx](#)
 - zeigt Streak-Badge bei einzelnen Habits mit Farblogik.
- [InputField.jsx](#)
 - generische Input-Feld-Komponente mit Label und Fehleranzeige.
- [Titel.jsx](#)
 - Header-Komponente zur Wiederverwendung
- Buttons in [components/ui/Buttons](#)
 - verschiedene Button-Komponenten:
 - [SaveButton.jsx](#)
 - [CloseButton.jsx](#)
 - [CalendarDayButton.jsx](#)
- Icons:
 - [DeleteIcon.jsx](#)
 - [EditIcon.jsx](#)

3.4.10. lib

- [convert.js](#): Datums-/Format-Utilities ([dateOnly](#), [monthAndYear](#), [todayAsString](#), [convertToGermanDateString](#), Berlin-Zeitzonevarianten, [isInFuture](#)).
- [calendar.js](#): Monatsmatrix, Tagesauswertung ([isEveryHabitChecked](#), [getCheckedHabitsFromDay](#), [getUncheckedHabitsFromDay](#)), [daysBetween](#).
- [habit.js](#): Statistiken ([getNumberOfCompletedHabits](#), [getNumberOfCompletedHabitsLastMonth](#), [getPercentageOfCompletedHabits](#), [getPercentageOfCompletedHabitsLastMonth](#)).

3.4.11. Tests ([_tests_](#))

- UI-, API- und Utility-Tests (siehe Verzeichnisstruktur oben).

3.5. Docker/Dev-Skripte (kurz)

- [docker-compose.yml](#): einfache Container-Definitionen (Ports, keine envs).
- [docker-compose.dev.yml](#): Dev-Setup mit Volumes und [backend/.env.local](#).
- [docker-compose.prod.yml](#): Prod-Setup mit [backend/.env.production](#) (falls vorhanden) und [front-](#)

end/.env.

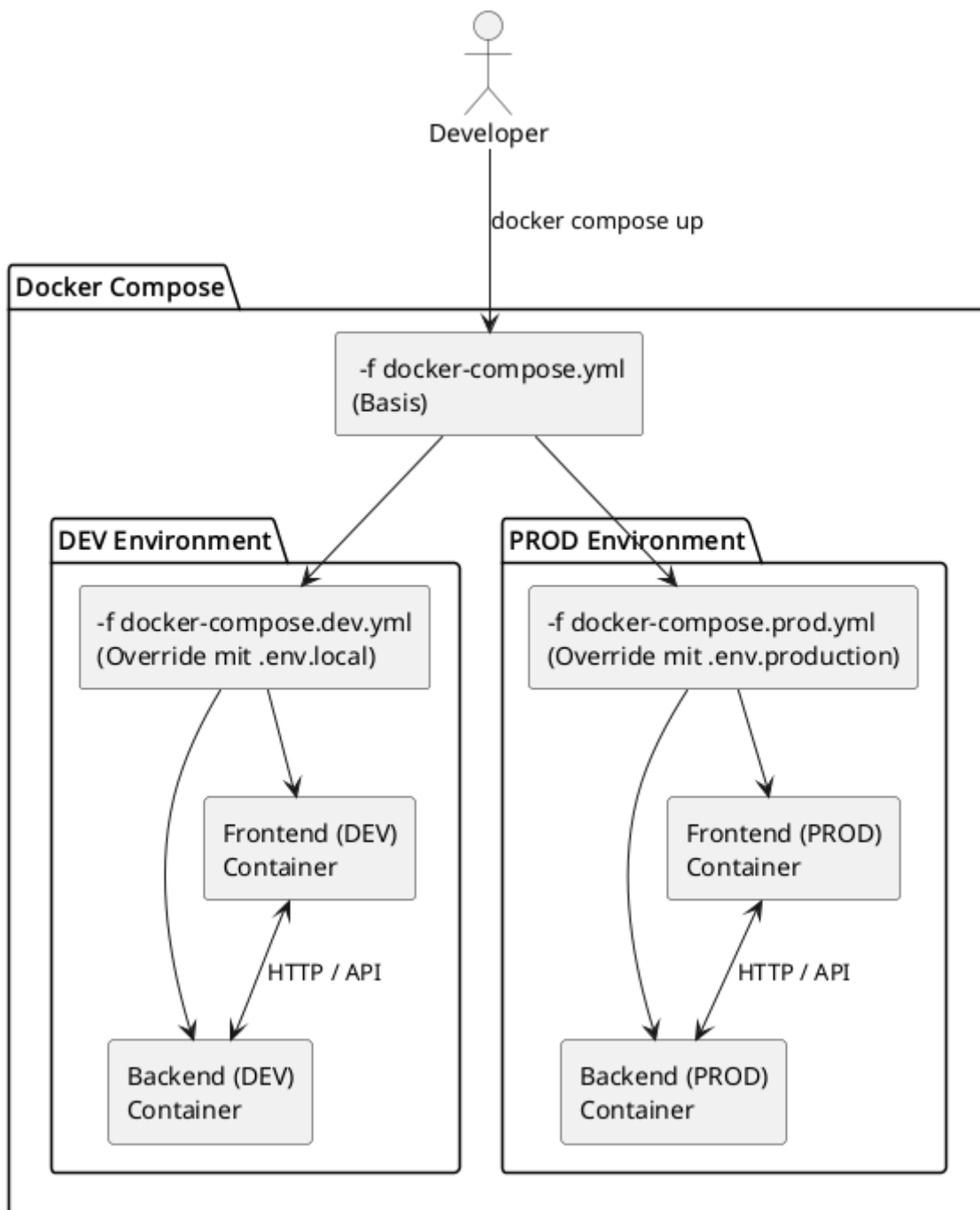
- `dev-start.` und `dev-db-setup.`: Hilfsskripte für lokalen Start und DB-Setup.

4. Setup und Guidelines

4.1. Docker Guide

4.1.1. Übersicht Containerstruktur

- **gewohnheitstier-backend**
 - Basierend auf `node:20-bullseye`
 - Enthält Express-Server, REST-API und Business-Logik
- **gewohnheitstier-frontend**
 - Basierend auf `node:20-bullseye`
 - Enthält React-SPA (CRA)



Die DEV-Umgebung kommuniziert dabei mit der lokalen Instanz von Supabase, während die PROD-Umgebung mit der Cloud-Instanz kommuniziert.

4.1.2. .env - Dateien

Die .env-Dateien setzen den nötigen Umgebungskontext via der entsprechenden Variablen. =====
env (Frontend)

```
REACT_APP_API_BASE_URL=http://localhost:3001
```

env.local

```
NODE_ENV=development
```

```
# Mac
#DATABASE_URL="postgresql://postgres:postgres@host.docker.internal:54322/postgres"
# Linux
DATABASE_URL="postgresql://postgres:postgres@172.17.0.1:54322/postgres"

# Mac
#SUPABASE_URL=http://host.docker.internal:54321
# Linux
SUPABASE_URL=http://172.17.0.1:54321

SUPABASE_ANON_KEY=****
PORT=3001
```

env.production

```
NODE_ENV=production
DATABASE_URL="postgresql://<user>.<project_ref>:g<password>@aws-0-eu-central-1.pooler.supabase.com:5432/postgres"
SUPABASE_URL=https://<project_ref>.supabase.co/
SUPABASE_ANON_KEY=****
PORT=3001
```

4.1.3. Nutzung

Projekt starten

```
docker compose up -f docker-compose.yml -f docker-compose.dev.yml up --build
```

bzw.

```
docker compose up -f docker-compose.yml -f docker-compose.prod.yml up --build
```

Dadurch werden beide Container gebaut und gestartet. Der Client ist anschließend unter <http://localhost:3000> und der Server unter <http://localhost:3001> erreichbar.

Projekt stoppen

```
docker compose down -v
```

-v entfernt dabei alle erstellten Volumes. Wichtig wenn Änderungen an den Nod_modules vorgenommen wurde.

Logs anzeigen

```
docker compose logs -f
```

Vorteile

- Einheitliche Entwicklungsumgebung für alle Teammitglieder
- Trennung von Entwicklung und Produktion
- Ermöglicht gleichzeitige Arbeit an Front- und Backend
- Leicht portierbar
- Isolation der Abhängigkeiten

Durch die Dockerisierung ist das Projekt **Gewohnheitstier** plattformunabhängig, reproduzierbar und teamfähig. Alle Entwickler können ohne komplexe Setups sofort starten.

4.2. Prisma ORM Guide

4.2.1. Überblick

Kurzer Leitfaden zur Prisma-Nutzung im aktuellen Projektstand.

4.2.2. Projektanbindung

- Schema: [backend/prisma/schema.prisma](#)
- Config: [backend/prisma.config.js](#) (Schema-Pfad, Migration-Ordner, Datasource-URL)
- Client: Standard-Output von `@prisma/client` (kein eigenes `output` gesetzt)
- Client-Import: [backend/src/prisma.js](#) erzeugt `PrismaClient` mit Logging und exportiert Instanz für das Backend (siehe [backend/index.js](#))
- DB-URL: `DATABASE_URL` aus [backend/.env](#) (lokal) bzw. [backend/.env.local](#) im Docker-Dev-Setup

4.2.3. Modelle im Schema

- Enums: `animal_type` (hund/katze/hamster/wurm), `animal_mood` (gluecklich/traurig)
- `habits_table`
 - Primärschlüssel `id` (String mit `uuid()` Default)
 - Felder u.a. `userId` (FK auf `users.id`), `habit_name`, `description`, `start_date`, `streak` (BigInt), `last_break`, `last_checked`, `prev_last_checked` (Array)
- `users`
 - Primärschlüssel `id` (String mit `uuid()` Default)
 - Felder für Tier-Status: `animal_type`, `animal_mood`
 - Relation: 1-n zu `habits_table`

4.2.4. Generierung & Migrationskommandos

Befehle innerhalb der Shell des Backend-Containers ausführen, nicht im Hostgerät:

- Client generieren (automatisch über Dockerfile oder in der Container Shell): `npx prisma generate`
- Schema validieren: `npx prisma validate`
- Migration erzeugen (falls benötigt): `npx prisma migrate dev --name <name>` (setzt lokale DB voraus)
- Schema ohne Migration in die DB pushen (leere DB): `npx prisma db push`
- Migrations chronologisch auf DB anwenden: `npx prisma deploy`
- Migrations liegen in [backend/prisma/migrations](#)

4.2.5. Nutzung im Code

- Beispiel-Query: [backend/src/routes/habits.js](#) Route `/api/habits` ruft `prisma.habits_table.findMany()` auf und serialisiert `streak/Dates` für JSON-Ausgabe.
- BigInt-Hinweis: Node JSON kann BigInt nicht serialisieren – deshalb werden `streak`-Werte in Numbers gewandelt (siehe `serializeHabit`).

4.2.6. Umgebungsvariablen

- `DATABASE_URL` muss vor Prisma-Aufrufen gesetzt sein.
 - Lokal: [backend/.env](#)
 - Docker-Dev: `backend/.env.local` (siehe `docker-compose.dev.yml`)
 - Docker-Prod: `backend/.env.production` (falls vorhanden, siehe `docker-compose.prod.yml`)
- Hinweis: `prisma.config.js` nutzt `env("DATABASE_URL")` aus Prisma Config; die `.env` wird nicht automatisch geladen, wenn die Umgebung nicht gesetzt ist.

4.2.7. Troubleshooting

- Schema-Änderungen greifen nicht: neu generieren, ggf. Migration/Push auf DB, eventuell fehlerhafte Migration zuvor erstellt die noch die "Pipeline" blockiert (aktuelle Migration ist gar nicht der Fehler) → reset auf `init_migration`
- Verbindung schlägt fehl: `DATABASE_URL` prüfen, Netzwerkzugriff zum DB-Host sicherstellen
- Wir arbeiten mit Prisma 6.x, darauf achten, dass nicht durch `npm install` auf Version 7.x geupdatet wurde

4.3. Supabase Guide

4.3.1. Überblick

Kurzer Überblick, wie Supabase aktuell genutzt wird: Auth läuft weiterhin über Supabase, aber

ausschließlich serverseitig. Das Frontend spricht nur die Express-API, die DB-Zugriffe übernimmt Prisma.

4.3.2. Clients & Umgebungsvariablen

- Backend: `SUPABASE_URL` und `SUPABASE_ANON_KEY` werden in `backend/src/supabaseAuth.js` genutzt, um gegen die Supabase Auth-REST-API zu authentifizieren. `DATABASE_URL` (Supabase-Postgres) bleibt für Prisma.
- Frontend: Kein Supabase-Client mehr. API-Basis per `REACT_APP_API_BASE_URL` wird in `.env` in `/frontend` gesetzt.

4.3.3. Auth-Flow (Backend-first)

- Auth-Endpunkte: `backend/src/routes/auth.js` kapselt `login`, `register`, `session` (Token-Validierung) und `logout` per Supabase-REST.
- Middleware: `backend/src/middleware/authenticate.js` validiert `Authorization: Bearer <token>` gegen Supabase und hängt `req.auth.user` an.
- Frontend-Integration: `frontend/src/api/authApi.js` ruft nur Backend-Endpoints auf, persistiert Token/User in `localStorage`. `App.jsx` lädt eine gespeicherte Session und reicht `session` an `HabitsPage` weiter.

4.3.4. Datenmodell (Supabase Tabellen)

- Tabelle: `habits_table` (siehe Prisma-Schema `schema.prisma`) mit Feldern `id` (String mit `uuid()`-Default), `userId` (String), `habit_name`, `description`, `start_date`, `streak`, `last_break`, `last_checked`, `prev_last_checked` (array/timestamptz).
- Tabelle: `users` mit `id` (String mit `uuid()`-Default) sowie `animal_type` und `animal_mood`.
- Erwartung: `userId` in `habits_table` entspricht Supabase-Auth-User-ID (`req.auth.user.id`).

4.3.5. Datenzugriff über das Backend

- Habit-Routen: `backend/src/routes/habits.js` implementiert `GET /api/habits`, `POST /api/habits`, `PUT /api/habits/:id`, `DELETE /api/habits/:id`, `POST /api/habits/:id/toggle` auf Basis von Prisma. Business-Logik (Toggle, Streak, Reset nach 48h) liegt im Backend.
- User-Routen: `backend/src/routes/users.js` implementiert `GET /api/users/animal` und `PUT /api/users/animal` für `animal_type/animal_mood`.
- API-Client im Frontend: `frontend/src/api/httpClient.js` baut Requests mit optionalem Bearer-Token; `frontend/src/api/habitsApi.js` und `frontend/src/api/userApi.js` konsumieren die Routen.
- UI: `frontend/src/pages/HabitsPage.jsx` ruft die API-Funktionen auf und reicht Daten an Grid/Kalender weiter.

4.3.6. Sicherheit / RLS

- Supabase Row Level Security weiterhin sinnvoll: Policies sollten `userId = auth.uid()` (bzw. den tatsächlichen Spaltennamen) erzwingen.

- Token-Verifikation passiert serverseitig; der Browser sieht keine DB-Credentials oder Supabase-Keys mehr.

4.3.7. Typische Fehlerquellen & Checks

- 401/403: Prüfe gültiges `Authorization`-Header zum Backend; Backend meldet Supabase-Fehler in `detail`.
- Leere Daten nach Login: `userId` der Zeilen muss mit Supabase-Auth-User übereinstimmen.
- BigInt im Backend: Serialisierung erledigt `serializeHabit` in `backend/src/routes/habits.js`; `streak` wird als Number zurückgegeben.
- Toggle-Logik läuft im Backend; falls alte Daten `prev_last_checked` nicht als Array vorliegen, prüfe DB-Einträge auf gültige Arrays.

4.4. ESLint Leitfaden (Frontend & Backend)

Dieser kurze Leitfaden erklärt, was Linting ist, warum es die Codequalität verbessert und wie wir ESLint in diesem Projekt nutzen.

4.4.1. Was ist Linting?

Linting analysiert Quellcode automatisch und erkennt typische Fehler, Stilbrüche und unsichere Muster. Vorteile:

- Einheitlicher Stil und bessere Lesbarkeit
- Frühe Fehlererkennung (z. B. unbenutzte Variablen, falsche Importe)
- Einhaltung von Best Practices (Framework-/Sprachregeln)
- Weniger Review-Aufwand und höhere Wartbarkeit

4.4.2. Voraussetzungen

- ESLint ist als Dev-Dependency installiert (Frontend und Backend)
- ESLint-Konfiguration wurde per `npx eslint --init` erzeugt (z.B. `eslint.config.js` oder `.eslintrc.*`)
- Container laufen für Befehle innerhalb von Docker

4.4.3. Lint ausführen

Frontend:

```
docker-compose -f docker-compose.dev.yml exec frontend npm run lint
```

Backend:

```
docker-compose -f docker-compose.dev.yml exec backend npm run lint
```

4.4.4. Automatische Korrekturen (Auto-Fix)

Viele Probleme lassen sich automatisch beheben:

Frontend:

```
docker-compose -f docker-compose.dev.yml exec frontend npm run lint:fix
```

Backend:

```
docker-compose -f docker-compose.dev.yml exec backend npm run lint:fix
```

4.4.5. Nützliche VS Code Einstellungen

Aktiviert automatische ESLint-Fixes beim Speichern (optional):

```
{
  "eslint.workingDirectories": [{ "mode": "auto" }],
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": true
  },
  "eslint.validate": ["javascript", "javascriptreact", "typescript",
"typescriptreact"]
}
```

4.4.6. Dateien/Ordner ignorieren

Erstellen/ergänzen von `.eslintignore` in Frontend und Backend (empfohlen):

```
node_modules
dist
build
coverage
.eslintcache
```

4.4.7. Tipps & Troubleshooting

- Container muss laufen, sonst schlägt `docker-compose exec ...` fehl: `docker-compose -f docker-compose.dev.yml up -d`
- Fehlende Konfiguration? Führe `npx eslint --init` im jeweiligen Service aus.
- Viele Warnungen? Nutze `lint:fix` und passe Regeln in der ESLint-Konfiguration an.

React (Frontend)

- Bei React ≥ 17 ist **React** in JSX-Dateien nicht mehr nötig. Deaktiviere die Regel:
- Installiere Plugins: [source,bash] ---- `docker-compose -f docker-compose.dev.yml exec frontend npm i -D eslint-plugin-react eslint-plugin-react-hooks ----`
- In der ESLint-Konfiguration: **react/react-in-jsx-scope: off** und **react/jsx-uses-react: off**
- Optional: Hooks-Regeln aktivieren (**react-hooks/rules-of-hooks**, **react-hooks/exhaustive-deps**)

ESLint v8/v9 Konfiguration

- Fehler wie „Package subpath './config' is not defined by exports ...“ deuten auf eine Flat-Config, die **eslint/config** importiert (nur in ESLint v9 vorhanden).
- Lösung:
- Auf ESLint v9 aktualisieren: `docker-compose -f docker-compose.dev.yml exec <service> npm i -D eslint@^9`

NPM-Skripte prüfen

- Stelle sicher, dass in **package.json** je Service folgende Skripte existieren:

```
{
  "scripts": {
    "lint": "eslint .",
    "lint:fix": "eslint . --fix"
  }
}
```

4.4.8. Was bringt Linting konkret?

- Klare, konsistente Codebasis an der sich das gesamte Team orientieren kann
- Schnellere Fehlerfindung vor Laufzeit/Build
- Höhere Code-Qualität und bessere Wartbarkeit langfristig
- Weniger Diskussionen über Stil im Code-Review

4.5. Styleguide

Dieses Dokument dient als Leitfaden für die Gestaltung von Frontend-Komponenten und die Einhaltung von Stilrichtlinien im Projekt *Gewohnheitstier*. Es soll sicherstellen, dass das Design konsistent, benutzerfreundlich und ansprechend ist. Das Projekt nutzt Tailwind CSS für das Design-System (Farben, Spacing, Typografie). Zusätzlich ist Bootstrap global eingebunden (Layout-Utilities/Grundstyles), daher können Bootstrap-Klassen einzelne Komponenten beeinflussen. Empfehlung: Wenn möglich, pro Komponente konsistent bleiben (Tailwind-only oder Bootstrap-only), um Style-Konflikte zu vermeiden.

4.5.1. Farbpalette & Semantik

Die Farbpalette von **Gewohnheitstier** basiert auf einer ruhigen, natürlichen Grün-Farbwelt. Ziel ist es, eine positive, motivierende Atmosphäre zu schaffen, die Gewohnheitsbildung unterstützt, ohne visuell zu überfordern.

Die Farben sind bewusst semantisch definiert und werden projektweit konsistent verwendet.

Farbe	Hex-Code	Tailwind-Klasse	Verwendung
Cremer	#FBF6E9	<code>bg-primary1 / text-primary1</code>	Flächen in Cards/Modals, Kontrastflächen auf dunkleren Hintergründen
Hellgrün	#E3F0AF	<code>bg-primary2 / text-primary2</code>	Standard-Seitenhintergrund, Hover-Flächen, ruhige Akzentflächen
Teal	#5DB996	<code>bg-primary3 / text-primary3</code>	Karten-/Icon-Flächen, sekundäre UI-Bereiche, visuelle Gruppierung
Dunkelgrün	#118B50	<code>bg-primary4 / text-primary4</code>	Primäre Akzente, aktive Zustände, Hervorhebungen, Fokus/Farbring
Rot	#FF6B6B	<code>bg-primaryRed / text-primaryRed</code>	Kritische Aktionen (Fehler, Warnung, Löschen)

Hinweis: Je nach Kontext können Cards sowohl auf `primary1` (hell, ruhig) als auch auf `primary3` (stärkerer Akzent, Fokuszustand) basieren, um visuelle Hierarchien zu unterstützen.

Kontrast & Zugänglichkeit

Zur Sicherstellung ausreichender Lesbarkeit gelten folgende Regeln:

- Auf hellen Hintergründen (`primary1`, `primary2`) wird bevorzugt **`primary4`** (#118B50) oder schwarzer Text verwendet.
- Auf dunkleren Flächen (`primary3`, `primary4`) wird **`primary1`** (#FBF6E9) als Textfarbe eingesetzt.
- **`primaryRed`** ist ausschließlich für kritische Aktionen (z. B. Löschen) oder Fehlermeldungen vorgesehen und darf nicht für reguläre Interaktionen verwendet werden.
- Zukünftige (nicht klickbare) Zustände werden neutral über Grautöne dargestellt (z. B. `bg-gray-300`), um sich klar von der Farbsemantik der App abzugrenzen.

4.5.2. Typografie

Die Typografie ist bewusst schlicht gehalten, um den Fokus auf Inhalte und Interaktionen zu legen. Es werden ausschließlich Tailwind-Utility-Klassen verwendet.

Element	Größe	Tailwind-Klasse
Hauptüberschrift	32px	<code>text-4xl font-bold</code>
Unterüberschrift (Seite)	24px	<code>text-2xl font-semibold</code>
Abschnittsüberschrift	20px	<code>text-xl font-semibold</code>

Element	Größe	Tailwind-Klasse
Body Text (Standard)	16px	<code>text-base</code>
Kleiner Text (Labels, Hinweise)	14px	<code>text-sm</code>
Sehr kleiner Text (Meta-Informationen)	12px	<code>text-xs</code>

Font-Familie

Das Projekt nutzt die Standard-Systemschriftart (System-Stack), um eine schnelle Ladezeit und plattformübergreifend konsistente Darstellung sicherzustellen.

```
font-family: ui-sans-serif, system-ui, -apple-system, "Segoe UI", Roboto, Arial, "Noto Sans", "Liberation Sans", sans-serif;
```

Abstandsskala

Das Projekt folgt einer konsistenten, 4px-basierten Spacing-Skala. Diese wird durchgängig für Padding, Margin und Grid-Abstände verwendet, um visuelle Ruhe und einheitliche Layouts zu gewährleisten.

Größe	Px	Tailwind-Klassen
xs	4px	<code>p-1, m-1, gap-1</code>
sm	8px	<code>p-2, m-2, gap-2</code>
md	16px	<code>p-4, m-4, gap-4</code>
lg	24px	<code>p-6, m-6, gap-6</code>
xl	32px	<code>p-8, m-8, gap-8</code>

Responsive Breakpoints

Das Frontend folgt einem **Mobile-First**-Ansatz. Komponenten werden zunächst für kleine Bildschirme gestaltet und anschließend für größere Viewports erweitert.

Gerät	Breakpoint	Tailwind-Präfix
Mobil	< 640px	<code>(no prefix)</code>
Tablet	sm: 640px	<code>sm:</code>
Desktop	lg: 1024px	<code>lg:</code>
Large Desktop	xl: 1280px	<code>xl:</code>

4.5.3. Komponenten-Katalog

Dieses Kapitel beschreibt die zentralen UI-Komponenten des Projekts **Gewohnheitstier**, ihre gestalterischen Merkmale sowie empfohlene Tailwind-CSS-Klassen zur konsistenten Umsetzung.

Buttons

Ziel: Buttons sollen klar verständlich, eindeutig unterscheidbar und barrierearm sein.

Merkmale:

- abgerundete Ecken (`rounded-lg` oder `rounded-xl`)
- ausreichender Innenabstand für gute Klickbarkeit
- klare Farbsemantik (Aktion vs. Gefahr)
- Hover- und Focus-Zustände für visuelles Feedback
- Hover bevorzugt über leichte Helligkeitsänderung (`hover:opacity-90`) statt starker Farbsprünge

Varianten:

Primärer Button (z. B. „Hinzufügen“, Bestätigen):

- Hintergrund: `primary4`
- Textfarbe: `primary1`
- Hover: `hover:opacity-90`

Empfohlen: `bg-primary4 text-primary1 px-4 py-2 rounded-xl font-medium shadow-sm hover:opacity-90 transition`

Hinweis: Primäre Buttons verwenden standardmäßig `primary4` (Bestätigen, Speichern). `primary3` kann für primäre, aber weniger kritische Aktionen (z. B. Navigation, Weiter) eingesetzt werden.

Sekundärer Button (z. B. Monat wechseln, Navigation):

- Hintergrund: `primary3` oder `primary2` (je nach Kontext)
- Textfarbe: `text-black` oder `text-primary4`
- Hover: leichte Abdunklung oder `hover:bg-primary2`

Empfohlen: `bg-primary3 text-black px-4 py-2 rounded-xl font-medium shadow-sm hover:bg-primary2 transition`

Destruktiver Button (z. B. Löschen):

- Farbe: `primaryRed`
- klare visuelle Abgrenzung von anderen Aktionen

Empfohlen: `bg-primaryRed text-white px-4 py-2 rounded-xl font-medium shadow-sm hover:opacity-90 transition`

Hinweis zur Wiederverwendung:

Wiederkehrende Buttons (z. B. „Schließen“-Button, Close-Icon in Modals, Bestätigungs-Buttons) sollen als wiederverwendbare UI-Komponenten umgesetzt werden. Bestehende Button-Komponenten sind bevorzugt zu verwenden, anstatt neue Varianten pro View zu definieren, um ein konsistentes Erscheinungsbild und wartbaren Code sicherzustellen.

Karten (Cards)

Cards sind generische Container-Komponenten und dienen als Grundlage für spezialisierte Varianten wie HabitCards oder Modals.

Merkmale:

- ruhiger Hintergrund mit leichtem Kontrast zum Seitenhintergrund
- abgerundete Ecken (iOS-ähnliche Anmutung)
- weicher Schatten für Tiefenwirkung
- klare visuelle Gruppierung von Inhalt

Empfohlen: `bg-primary1 rounded-2xl shadow-md p-4 border border-gray-200`

HabitCard (Gewohnheits-Kachel)

Die HabitCard stellt eine einzelne Gewohnheit dar. Sie ist als horizontale Listen-Card gestaltet und stellt eine einzelne Gewohnheit kompakt dar.

Merkmale:

- horizontales Layout
- Status-Toggle links (Erledigt / Nicht erledigt)
- Inhalt mittig (Name, optionale Beschreibung)
- Meta-Informationen rechts (Streak, Bearbeiten, Löschen)
- gesamte Zeile klickbar
- kompakt, aber gut touch-/klickbar

Gestaltung:

- Hintergrund: `primary3`
- Text: `primary1`
- Akzente (Icons / Streak): `primary1` oder `primary4`

Empfohlen: `bg-primary3 rounded-xl shadow-sm px-3 py-2 flex items-center justify-between hover:shadow-md transition`

Grid / Layout (HabitGrid)

Das HabitGrid ordnet mehrere HabitCards übersichtlich an.

Merkmale:

- Habits-Bereich oben (Liste)
- Kalender darunter (zentriert)
- Mobile-first sowieso untereinander

Empfohlen:

- Page-Wrapper: `max-w-5xl mx-auto px-4`
- Stack: `flex flex-col gap-6` (oder `gap-8`)

Kalender

Der Kalender fungiert zugleich als visuelles Feedback-Instrument zur Motivation und langfristigen Gewohnheitsverfolgung. Nicht klickbare Tage (Zukunft) sind bewusst visuell und funktional deaktiviert, um Fehlinteraktionen zu vermeiden.

Merkmale:

- klare Trennung zwischen Vergangenheit, Gegenwart und Zukunft
- vergangene Tage: weiß
- heutiger Tag: hervorgehoben (Ring oder Rahmen)
- zukünftige Tage: grau und deaktiviert
- visuelles Feedback für Erledigungsgrad (Punkt oder Progress-Ring)

Farbsemantik:

- Vergangenheit: `bg-primary1`
- Heute: `ring-2 ring-primary4`
- Zukunft: `bg-gray-300 text-gray-500`

Empfohlen: `grid grid-cols-7 gap-2`

Modal (Dialogfenster)

Modals werden für Detailansichten verwendet (z. B. Kalender-Tag, Habit-Details).

Merkmale:

- dunkler Hintergrund mit Blur-Effekt
- zentrierte Card mit Fokus auf Inhalt
- Schließen über X oben rechts und Button im Footer
- klare visuelle Hierarchie (Header / Content / Footer)
- dezente Umrandung zur Trennung vom Overlay

Overlay: `fixed inset-0 bg-black/40 backdrop-blur-sm flex items-center justify-center z-50`

Modal-Card: `max-w-2xl w-full bg-primary1 rounded-2xl shadow-2xl border border-gray-200 overflow-hidden`

Formulare & Inputs

Formulare sollen leicht, klar und gut lesbar sein.

Merkmale:

- Eingabetext immer dunkel (hohe Lesbarkeit)
- Placeholder neutral grau
- Fokuszustand in App-Farbwelt (`primary4`) statt Default-Blau

Empfohlen (Input/Textarea): `border border-slate-200 bg-slate-50/80 text-slate-900 placeholder:text-slate-400 rounded-xl px-3 py-2.5 focus:border-primary4 focus:ring-2 focus:ring-primary4/20 focus:bg-white transition outline-none`

Navbar

Die Navbar bleibt bewusst zurückhaltend, um den Fokus auf Inhalte und tägliche Interaktionen zu legen.

Merkmale:

- ruhiges, helles Farbschema
- aktive Links farblich hervorgehoben
- keine Bootstrap-Buttons
- Logo als Kreis-Icon

Gestaltung:

- Hintergrund: `primary2`
- Aktiver Link: `primary4`
- Inaktive Links: Grauton

Empfohlen: `bg-primary2 border-b border-gray-200`

Typografie & Textelemente

Merkmale:

- klare Hierarchie
- gute Lesbarkeit auf hellen Hintergründen
- sparsame Nutzung von Akzentfarben

Empfohlen:

- Überschriften: `text-primary4` oder `text-black` (je nach Kontext)
- Fließtext: `text-gray-700`
- Hinweise/Meta: `text-gray-500 text-sm`

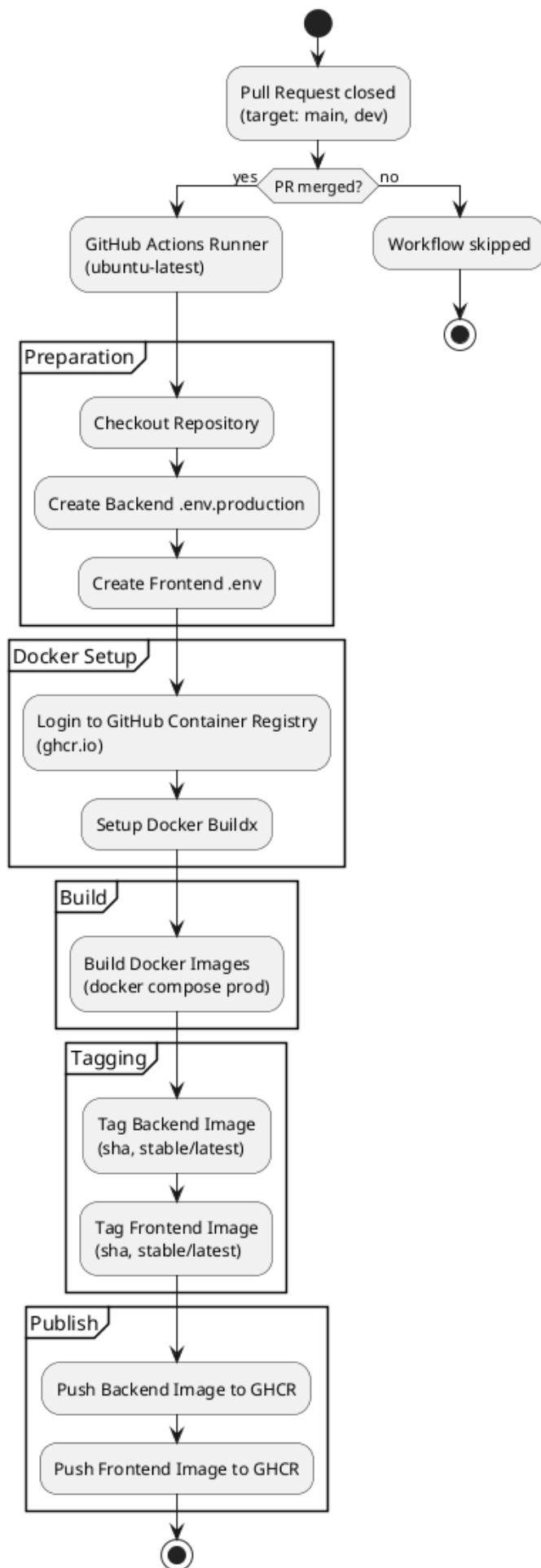
5. Integration und Deployment

5.1. Integration

5.1.1. GitHub Actions Workflow - ci.yml

- siehe [.github/workflows/ci.yml](#)

Docker Image Deployment Workflow



5.2. Deployment

5.2.1. GitHub Actions Workflow - docker-deploy.yml

- siehe [.github/workflows/docker-deploy.yml](#)

Das Image-Tag ist abhängig vom Target-Branch, d.h. bei einem Merge auf den main-Branch wird das Image mit **stable** getaggt, bei einem Merge auf den dev-Branch mit **latest**. Dies ermöglicht uns mehrere Versionen gleichzeitig verwalten zu können.

Docker Image Deployment Workflow

